Surrogate-Assisted Differential Evolution with Adaptive Multi-Subspace Search for Large-Scale Expensive Optimization

Haoran Gu, Handing Wang, *Member, IEEE,* and Yaochu Jin, *Fellow, IEEE*

***Abstract*—Real-world industrial engineering optimization problems often have a large number of decision variables. Most existing large-scale evolutionary algorithms need a large number of function evaluations to achieve high-quality solutions. However, the function evaluations can be computationally intensive for many of these problems, particularly, which makes large-scale expensive optimization challenging. To address this challenge, surrogate-assisted evolutionary algorithms based on the divide-and-conquer strategy have been proposed and shown to be promising. Following this line of research, we propose a surrogate-assisted differential evolution algorithm with adaptive multi-subspace search for large-scale expensive optimization to take full advantage of the population and the surrogate mechanism. The proposed algorithm constructs multi-subspace based on principal component analysis and random decision variable selection, and searches adaptively in the constructed subspaces with three search strategies. The experimental results on a set of large-scale expensive test problems have demonstrated its superiority over three state-of-the-art algorithms on the optimization problems with up to 1000 decision variables.**

***Index Terms*—Multi-subspace search, adaptive search switching strategy, large-scale expensive optimization, radial basis function network, surrogate.**

I. Introduction

MOST real-world industrial engineering optimization problems have high-dimensional decision variables [1]–[3]. The evaluations of these problems generally rely on time-consuming numerical simulations, where each evaluation takes from minutes to hours, and even days [4]–[7]. When the decision variables of the problem are large-scale, the required computation cost increases further because of the exponential growth of the search space. Unfortunately, due to the high cost of each evaluation, the computation budget is often far from enough, which makes large-scale expensive optimization challenging.

In recent years, evolutionary algorithms (EAs) [8] with a global search capability and better robustness have been proposed to solve large-scale optimization problems, which assume that objective functions are available and cheap for quality evaluations, so that thousands of evaluations can be done to achieve high-quality solutions of the problem [7]. It has been proved that if the number of function evaluations is enough, the quality of these solutions is acceptable [9]–[12]. Existing large-scale EAs can be roughly divided into the following two categories: decomposition-based and non-decomposition-based approaches [13]. The decomposition-based methods are inspired by the divide-and-conquer strategy. They decompose the original large-scale decision space into multiple low-dimensional subcomponents which are simple enough to be solved independently and iteratively, finally combine the subcomponents to form the solution of the original problem. The cooperative coevolutionary (CC) framework [14] is a well-known decomposition-based method, whose decomposition idea is embodied in grouping large-scale decision variables and optimizing these groups respectively. It has been proved [15], [16] that the correlation between variables will greatly affect the performance of optimization algorithms in a continuous domain. In the CC framework, if the correlated decision variables are not grouped together, the epistatic interaction which can negatively reduce the convergence speed may occur during the optimization process [14], [17], [18]. Consequently, an effective grouping strategy is extremely important in the CC framework. The current grouping strategies are as follows: random grouping [11], recursive differential grouping [10] and differential grouping [16]. In particular, to place all correlated decision variables into the same group, the latter two grouping strategies require function evaluations to detect the correlation between variables. In the non-decomposition-based categories, the original problem is directly optimized through enhanced optimization strategies [19]. These enhanced optimization strategies can accelerate the convergence of the algorithms. For example, the competitive swarm optimization (CSO) [9] introduces a pairwise competition mechanism, where the velocity of the particle that loses the competition will be updated by learning from the winner particle. The social learning particle swarm optimization algorithm (SL-PSO) [20] applies a social learning mechanisms into particle swarm optimization (PSO) [21], specifically, each particle randomly learns from any better particles (termed demonstrators) in the current swarm. The level-based learning swarm optimizer (LLSO) [12] employs a level-based learning strategy, which separates particles into a lot of levels according to their fitness values and treats particles differently in different levels. Then, it randomly selects two

This work was supported in part by the National Natural Science Foundation of China (No. 61976165). Y. Jin is endowed an Alexander von Humboldt Professorship for Artificial Intelligence funded by the German Federal Ministry of Education and Research. (*Corresponding author: Handing Wang*)

H. Gu and H. Wang are with School of Artificial Intelligence and the Collaborative Innovation Center of Quantum Information of Shaanxi Province, Xidian University, Xi'an 710071, China. (e-mail: xdu_guhaoran@163.com, hdwang@xidian.edu.cn).

Y. Jin is with the Chair of Nature Inspired Computing and Engineering, Faculty of Technology, Bielefeld University, D-33619 Bielefeld, Germany. He is also with the Department of Computer Science, University of Surrey, Guildford, GU2 7XH, U.K. (e-mail: yaochu.jin@uni-bielefeld.de).

predominant particles from two different higher levels in the current swarm to direct the learning of particles.

Nevertheless, as mentioned above, when the optimization problem is expensive, there are not enough function evaluations to locate the global optimum. In order to solve such problems, a number of surrogate models have been used to replace some expensive function evaluations in EAs, such as polynomial regression (PR) [22], Gaussian processes (GP) [23], radial basis function networks (RBFN) [24], neural networks (NN) [25] and support vector machines (SVMs) [26], [27]. This surrogate mechanism allows EAs to avoid using too many expensive function evaluations. Those EAs are called surrogate-assisted evolutionary algorithms (SAEAs) [28]–[32]. SAEAs have shown better performance than traditional EAs on expensive problems with up to 200 decision variables [7]. Unfortunately, as the number of decision variables increases, global surrogate models easily become inaccurate, which degenerates the performance of SAEAs. Recently, a number of SAEAs try to address this hardness. For example, a global and local surrogate-assisted differential evolution (DE) algorithm (GL-SADE) [33] is proposed for solving high-dimensional expensive problems, in which a global RBFN is trained with all samples to accelerate convergence with the help of its current optimum, and a local Kriging model is used to modify the search area to reach the global optimum. Li *et al.* [34] utilized two swarms to collaboratively explore and exploit the search space with a dynamic swarm size adjustment scheme. In [35], the global exploration ability of DE and a local surrogate model are combined to solve high-dimensional expensive problems. However, the above SAEAs can only handle the 200-dimensional problems at most.

So far, a small number of methods have been proposed for expensive optimization problems with more than 200 decision variables, and these proposed methods can be roughly classified into two different categories: 1) enhanced non-surrogate-assisted optimizers; 2) surrogate-assisted optimizers based on the divide-and-conquer strategy. In the first category of optimizers, there is no surrogate mechanism. For example, Rahnamayan *et al.* proposed a modified coordinate descent algorithm (MCD) [36], which can shrink the search space by folding it into the half with exponential speed. Unfortunately, when the extreme point is near the boundary of the solution space, the optimization performance will become very poor due to the way of shrinking the search space. In addition, the researchers also attempt to assist optimizers with the fitness approximation strategy, e.g., the fitness approximation assisted competitive swarm optimizer (FAACSO) [37] estimates the fitness based on the positional relationship between individuals instead of a surrogate model in CSO. Therefore, FAACSO significantly reduces the number of function evaluations. Since it estimates the fitness only based on the positional relationship between individuals, it is insensitive to dimensions of decision variables and has good robustness. However, its performance depends heavily on the searching ability of CSO itself. The second category adopts the generic surrogate models to assist the optimizers based on the divide-and-conquer strategy. Specifically, this category builds a surrogate model for each decomposed subcomponent to replace most expensive function evaluations. For example, the surrogate-assisted cooperative coevolutionary (SACC) algorithm [18] creates non-overlapping variable groups of the same dimension randomly and then uses the surrogate models to assist the optimization process inside every group. SACC has successfully achieved the dimension reduction of large-scale decision variables and requires a relatively small number of function evaluations. Unfortunately, due to the traditional random grouping strategy in the SACC algorithm, there is not much chance when all correlated variables fall into the same group, thus results in the poor convergence. To address the above weaknesses in the SACC algorithm, a surrogate-assisted evolutionary algorithm with random feature selection (SAEA-RFS) [19] introduces the following modification: the overlapping subcomponents, the greedy mechanism of updating the global optimal solution. With the above improvements, SAEA-RFS can increase the probability that part of correlated decision variables are grouped together. It can also make efficient use of the limited expensive function evaluations, because the expensive function evaluations are only used for evaluating the obtained optimal solution of each generation in the optimization process. Therefore, SAEA-RFS has good performance in solving large-scale expensive optimization problems. However, in SAEA-RFS, each decision variable is selected with the same probability to form a sub-problem. The population search information is not considered to accelerate the convergence. In addition, since each subcomponent is constructed by random feature selection, the probability that all correlated decision variables are grouped together is extremely low, which will negatively affect the optimization efficiency. To make subcomponent construction more efficient and take full advantage of population search information, we propose a surrogate-assisted DE algorithm with an adaptive multi-subspace search (SADE-AMSS) for large-scale expensive optimization. The algorithm constructs subspaces in the mapping space based on the information of past population search. Then a series of surrogate models which contain the information of the whole decision space are established to assist the search in the mapping subspaces. Meanwhile, the algorithm also constructs subspaces in the original space to improve the exploration ability in the decision space. Hence, a local promising area is quickly located and the convergence speed is increased greatly. According to the optimization stage, an adaptive search switching strategy is proposed to keep a good balance between the exploration and the exploitation. The main contributions of this paper are as follows:

- We propose a method based on principal component analysis (PCA) and random decision variable selection to construct multiple subspaces for optimization, which aims to take full advantage of the population and the surrogate mechanism, thus further speed up the optimization process.
- Since the searching processes in the mapping subspaces and the original subspaces have different behaviors, we propose three search strategies with different searching ratios of those two kinds of subspaces. In addition, an adaptive switch is applied according to the optimization

stage to meet the requirement of both surrogate models and optimization.

The remainder of this paper is organized as follows: In Section II, an introduction to RBFN, PCA, SAEA-RFS is given. Section III gives a detailed description of the proposed algorithm. In Section IV, the experimental results and analysis are given. Finally, Section V summarizes the paper and provides the future work.

II. Preliminaries

A. Radial Basis Function Network

RBFN [38] is a feedforward neural network with three layers: an input layer, a hidden layer and an output layer. Due to its satisfactory performance on high-dimensional nonlinear problems [39], RBFN is often adopted as the surrogate model in solving large-scale problems [18], [19].

Different from common three-layer neural networks, the hidden layer of RBFN uses radial basis function (RBF) to nonlinearly map the input to the output. The RBF can be calculated by the following equation:

$$\Phi(r) = \Phi(\|\mathbf{x} - \mathbf{c}\|), \tag{1}$$

where $r = \|\mathbf{x} - \mathbf{c}\|$ means the distance from the center point $\mathbf{c}$. Φ means various kernels in RBF, and in this paper, we adopt the cubic function [40] because it is simple and there are no extra parameters that need to be regulated. The cubic function is as shown in Equation (2).

$$\Phi(r) = r^3. \tag{2}$$

The output of RBFN is the combination of RBF and neuron parameters in the network, and it can be calculated by the following equation:

$$\begin{aligned} Output &= \sum_{i=1}^{n} \omega_i \Phi(\|\mathbf{x} - \mathbf{c}_i\|) + bias, \\ s.t. \quad \mathbf{x} &= (x_1, ..., x_d)^T, \end{aligned} \tag{3}$$

where ω_i represents the weight between the i-th hidden neuron and the output neuron, n is the number of the center points $\mathbf{c}$, $\Phi(\|\mathbf{x} - \mathbf{c}_i\|)$ is the output of the i-th hidden neuron, and d is the dimension of $\mathbf{x}$. $bias$ can be expressed as a linear combination of inputs $\mathbf{x} = (x_1, ..., x_d)^T$ and a constant x_0 [33] [40], which is also known as a polynomial tail. It can be expressed by the following equation:

$$bias = \sum_{j=1}^{d} \lambda_j x_j + x_0, \quad \lambda_1, ..., \lambda_d \in \mathbb{R}, \tag{4}$$

where λ_j is the parameter of the optional polynomial tail.

B. Principal Component Analysis

The traditional feature selection method selects effective dimensions for data dimension reduction according to the prior information of data such as variance, pearson coefficient, mutual information, Gini coefficient of decision tree, etc. Inspired by it, in the absence of any prior information, the random feature selection technique of SAEA-RFS is an effective way to reduce the dimension of large-scale decision variables. Different from the above dimension reduction method, PCA [41] analyzes the distribution of data, and the analyzed results are extracted to form the mapping matrix $\mathbf{M} \in \mathbb{R}^{D \times d}$.

Consider the variable $\mathbf{x}$ with dimensionality D, for any $\mathbf{x} \in \mathbb{R}^D$, there exists with probability 1 a solution $\mathbf{y} \in \mathbb{R}^d$, it can be shown such that

$$(\mathbf{x} - \mathbf{x}_{mean})\,\mathbf{M} = \mathbf{y}, \tag{5}$$

where $\mathbf{M}$ represents the eigenvector matrix of the input data after removing the mean value, in which, the eigenvectors are sorted according to the magnitude of their corresponding eigenvalues. $\mathbf{x}_{mean}$ is the mean vector of the input data.

In conclusion, feature selection reduces dimension by selecting a sub-set of the original decision variables. PCA realizes the dimension reduction by linear mapping from a high-dimensional coordinate system to a low-dimensional coordinate system. The mapping variables are linear combinations of all the original decision variables with different weight matrices.

C. Surrogate-Assisted Evolutionary Algorithm with Random Feature Selection

SAEA-RFS [19] is a surrogate-assisted algorithm based on the divide-and-conquer strategy, which was proposed to solve large-scale expensive optimization problems. The main idea of SAEA-RFS is utilizing a random feature selection technique to select partial decision variables to form a series of sub-problems for serial optimizing. In SAEA-RFS, DE [42] is chosen as the optimizer. Concretely, $DE/best/1$ and $binomial\ crossover$ are used as mutation and crossover strategy to generate new individuals. As shown in Fig. 1, the framework of SAEA-RFS can be summarised as follows.

(1) Initialization : Initialize a population for optimization. Generate N_s individuals using the Latin hypercube sampling, evaluate their fitness using the exact expensive function, and save them in an archive Arc that stores the training set for the surrogate models. k represents the k-th sub-problem, set $k = 1$. K is the number of sub-problems in a generation.

(2) Sub-problem Generation : Generate an integer number within $[1, D_{\max}]$, and denote it as D_{sp_k}, then randomly select D_{sp_k} decision variables to form a sub-problem sp_k out of order. The decision space of sp_k is denoted as:

$$\mathbf{x}^{sp_k} = (x_1, x_2, ..., x_{D_{sp_k}}). \tag{6}$$

The population of sp_k is shown as:

$$\mathbf{pop}^{sp_k} = ({\mathbf{x}_1}^{sp_k}; {\mathbf{x}_2}^{sp_k}; ...; {\mathbf{x}_N}^{sp_k}), \tag{7}$$

where $D_{\max}$ represents the preordained maximum dimension of all sub-problems, N is the number of individuals in the population.

(3) Surrogate Model Training : Train an RBFN for sp_k using the training set from Arc, denoted as M_k. The traing set is denoted as:

$$\begin{aligned} {\mathbf{X}_{train}}^{sp_k} = \{&{\mathbf{x}_1}^{sp_k}, f(\mathbf{x}_1); \\ &{\mathbf{x}_2}^{sp_k}, f(\mathbf{x}_2); ...; {\mathbf{x}_{tsn}}^{sp_k}, f(\mathbf{x}_{tsn})\}, \end{aligned} \tag{8}$$

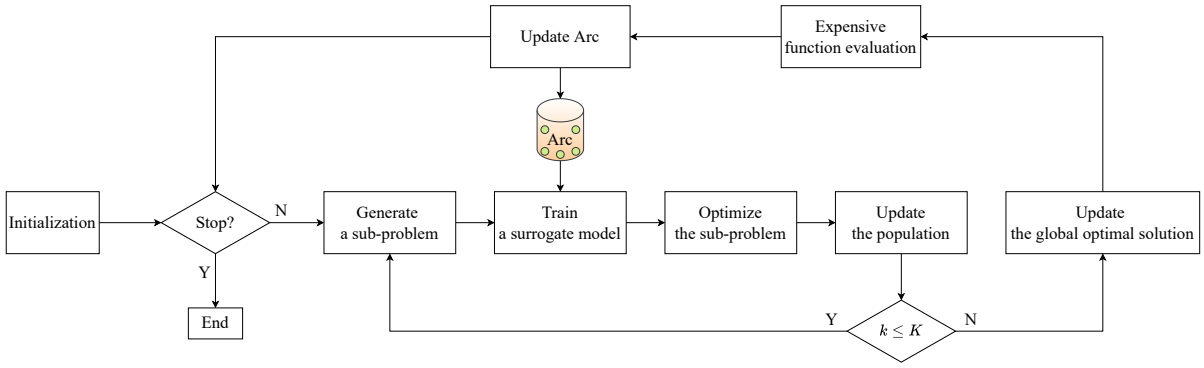


Fig. 1. Flowchart of SAEA-RFS.

where tsn represents the number of individuals in the training set, $f(\mathbf{x}_{tsn})$ represents the exact function value of $\mathbf{x}_{tsn}$.

(4) Sub-problem Optimization : Generate new individuals using DE, which can be expressed by the following equations:

1) Mutation

$$\begin{aligned} {\mathbf{v}_i}^{sp_k} &= {\mathbf{x}_{best}}^{sp_k} + F \cdot ({\mathbf{x}_{r_1}}^{i\,sp_k} - {\mathbf{x}_{r_2}}^{i\,sp_k}), \\ s.t. \quad & i = 1, 2, ..., N, \end{aligned} \tag{9a}$$

where ${\mathbf{v}_i}^{sp_k}$ is the i-th newly generated individual, $\mathbf{x}_{best}$ is the best individual with the best fitness in Arc. The indexes ${r_1}^i$ and ${r_2}^i$ are mutually exclusive integers randomly chosen from the range $[1, N]$, and all are different from the base index i. F is the mutation scaling factor.

2) Crossover

$$\begin{aligned} v_{j,i} &= \begin{cases} v_{j,i}, & if(rand[0,1] \leq CR \ \ or \ \ j = j_{rand}) \\ x_{j,i}, & otherwise \end{cases}, \\ & s.t. \quad j = 1, 2, ..., D_{sp_k}, \end{aligned} \tag{9b}$$

where $v_{j,i}$ is the j-th component of ${\mathbf{v}_i}^{sp_k}$, $x_{j,i}$ is the j-th component of ${\mathbf{x}_i}^{sp_k}$, $j_{rand} \in [1, 2, ..., D_{sp_k}]$, CR is the crossover rate.

Approximate these individuals by M_k, and then select the best N individuals from the original individuals and the newly generated individuals to enter the next iteration of DE. After the optimization, select the better N individuals to form $\mathbf{pop}^{sp_k}$. Find the optimal solution ${\mathbf{x}_b}^{sp_k}$.

$$\begin{aligned} {\mathbf{x}_b}^{sp_k} = \arg\min\{ & \hat{f}^{M_k}({\mathbf{x}_1}^{sp_k}), \\ & \hat{f}^{M_k}({\mathbf{x}_2}^{sp_k}), ..., \hat{f}^{M_k}({\mathbf{x}_N}^{sp_k})\}, \end{aligned} \tag{10}$$

where $\hat{f}^{M_k}(\mathbf{x}^{sp_k})$ is the fitness value of $\mathbf{x}^{sp_k}$ approximated by its surrogate model M_k.

(5) Population Update : Replace the decision variables of population on the corresponding dimension of sp_k using optimized $\mathbf{pop}^{sp_k}$. Then let $k = k + 1$, and if $k \leq K$, go back to step (2).

(6) Global Optimal Solution Update : Among all sub-problems, find the optimal solution, i.e.,

$$\begin{aligned} \mathbf{x}_{bg} = \arg\min\{ & \hat{f}^{M_1}({\mathbf{x}_b}^{sp_1}), \\ & \hat{f}^{M_2}({\mathbf{x}_b}^{sp_2}), ..., \hat{f}^{M_K}({\mathbf{x}_b}^{sp_K})\}. \end{aligned} \tag{11}$$

The sub-problem of $\mathbf{x}_{bg}$ is denoted as sp_i, $1 \leq i \leq K$. Then approximate the fitness of solutions $\mathbf{pop}^{sp_i}$ using the corresponding surrogate model M_i, the optimal solution is denoted as $\mathbf{x}_{bp}$, and if $\hat{f}^{M_i}(\mathbf{x}_{bg}) < \hat{f}^{M_i}(\mathbf{x}_{bp})$, replace the corresponding decision variables of the current global optimal solution using $\mathbf{x}_{bg}$, else replace the corresponding dimensions of the current global optimal solution using $\mathbf{x}_{bp}$. Next, evaluate the fitness of the new generated global optimal solution using the exact expensive fitness function and update the archive Arc.

(7) Stopping Condition : Determine whether the stopping condition is met, if not, return to step (1).

III. Proposed Algorithm

In this work, an SADE-AMSS algorithm based on the divide-and-conquer strategy is proposed for large-scale expensive optimization. In SADE-AMSS, the large-scale variables will be decomposed into the multiple overlapping subcomponents (termed subspaces), each subspace will be optimized sequentially assisted by a surrogate model. To make subspace construction efficient, we propose the idea of multi-subspace search in SADE-AMSS. DE forces the population to move gradually towards the local optimal area. The population distribution in the decision space can show the search information. To be specific, based on the population distribution, the algorithm will not have much progress on the search direction with a small variance, while it has not been fully exploited on the direction with a large variance. Therefore, greedily, assigning much search resource on the direction with a large variance can accelerate convergence.

It is often that each of those promising directions is a linear combination of decision variables, which can be extracted by PCA. As the example in Fig. 2 shows, y_1 represents the direction corresponding to the maximum variance between the individuals mapped on this direction. x_1 is a randomly chosen direction in the original decision space. It is clear that y_1 has stronger exploitation potential than x_1. We select the variables corresponding to the directions such as y_1 to form the subspace for optimization, so that DE can converge to a local promising area quickly. However, in the high-dimensional decision space, those directions are always different from the corresponding directions of large-scale decision variables. Therefore, we construct the mapping space based on the current population

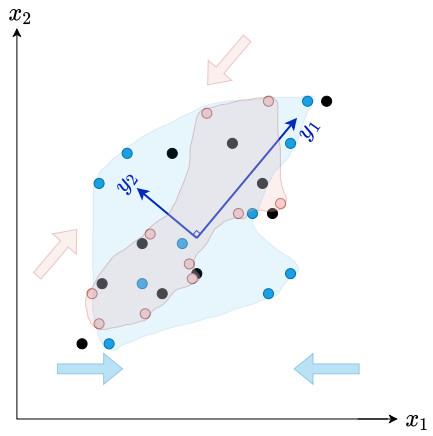


Fig. 2. Schematic comparisons of the region of the population generated by DE on y_1 with the region of the population generated by DE on x_1. The black circles represent the original population. The blue circles represent the population generated on x_1. The pink circles represent the population generated on y_1.

distribution through PCA to extract those directions to speed up the search.

Moreover, in the constructed mapping space, each variable is a linear combination of all decision variables in the original space. Therefore, optimizing the variables of the mapping space can address the major hardness in the decomposition-based methods, that is, how to group the correlated decision variables together for optimization, because each variable is equivalent to a group which contains all correlated decision variables of the original space. The different combination weights of the different variables in the mapping space represent the differences between groups. In addition, each variable in the mapping subspaces integrates the information about all the original decision variables that directly affect the exact function value. We use the mapped training sets in the mapping subspaces and the corresponding exact function values to build the surrogate models. This surrogate modeling mechanism which contains the information of the whole decision space, is promising for the global search at the beginning of the optimization process. Whereas, in the original space, the surrogate models are trained directly by the decision variables in the subspaces.

Furthermore, as presented in Fig. 2, the generated population on the direction of mapping space y_1 spreads smaller than x_1 of the original space. The search in the mapping subspaces contributes to the local exploitation, while the search in the original subspaces contributes to the global exploration. In order to keep a good balance between the exploration and the exploitation, we divide the optimization process into three stages, and configure a search strategy for each stage. The schematic diagram of our algorithm is shown in Fig. 3. SADE-AMSS searches in the multiple subspaces constructed in the original space and the mapping space. With these three search strategies that can be switched adaptively, SADE-AMSS can effectively handle large-scale expensive optimization problems. SADE-AMSS can be further divided into eight main steps.

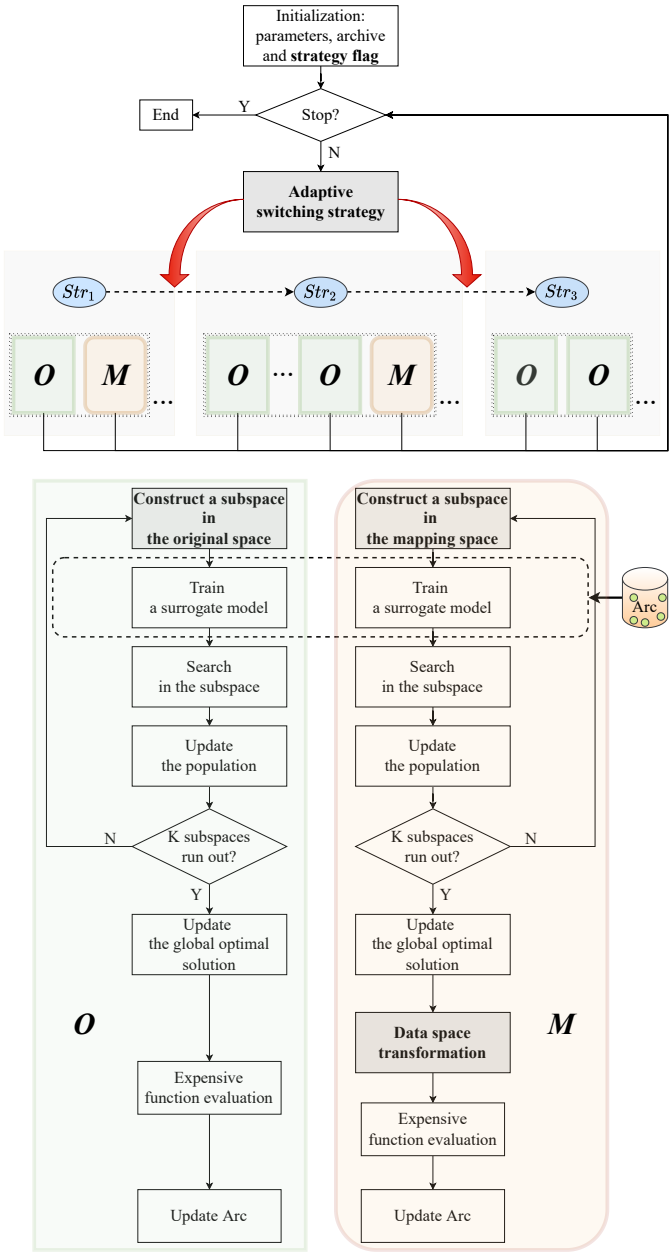


Fig. 3. Framework of the proposed SADE-AMSS.

1) *Initialization:* Initialize a population of N individuals for optimization. Generate N_s individuals using the Latin hypercube sampling, evaluate their fitness using the exact expensive function, and save them in an archive Arc that stores the training set for the surrogate models. In addition, the strategy flag is set to Str_1. More details can be found in Section III-*B*.
2) *Adaptive Switching Strategy:* Calculate an indicator DSD to set the strategy flag.
3) *Subspace Construction:* Select one space either the original space or the mapping space according to the strategy flag Str_k. Generate a random number d representing the dimension of the subspace. Construct a subspace in the selected space.
4) *Surrogate Model Training:* Randomly select tsn samples

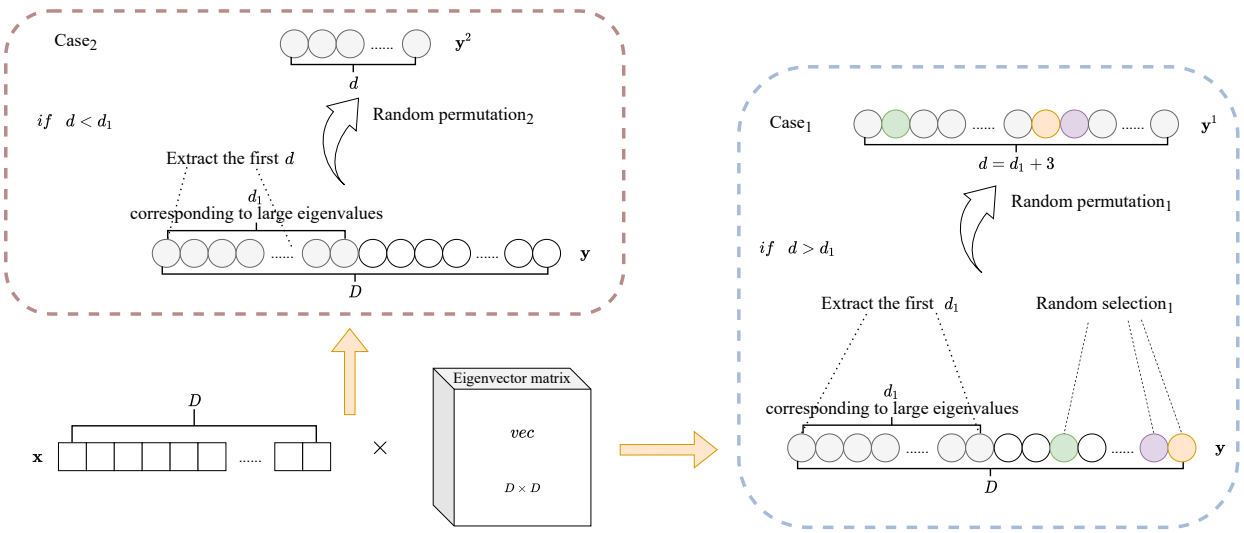


Fig. 4. Construction process of two mapping subspaces based on PCA.

from Arc to form the original training set $\mathbf{X}_{train}$, train an RBFN for the subspace.

5) *Subspace Search:* Search in the subspace by DE, and approximate the individuals generated by DE using the surrogate model trained in the previous step. Then select the better N individuals from the original individuals and the newly generated individuals to enter the next iteration of DE. After the optimization, select the better N individuals to form the optimized population.
6) *Population Update:* Replace the variables of population on the corresponding dimension using optimized population in the subspace. Then go back to 3) until K subspaces run out.
7) *Global Optimal Solution Update:* Similar to Step (6) in the framework of SAEA-RFS, the mechanism of updating the global optimal solution is greedy. In particular, after the new global optimal solution is generated, if the space is the mapping one, the population and the new optimal solution need to be transformed to the original space. Then use expensive function to evaluate the global optimal solution, and update the archive Arc.
8) Repeat steps 2)-7) until the computation budget runs out.

To describe conveniently, a global optimal solution updating process is denoted as a generation. In the following sections, we will present the details of the multi-subspace construction and adaptive search switching strategy.

A. Multi-Subspace Construction

In SADE-AMSS, subspaces are constructed based on PCA in the mapping space, and random decision variable selection in the original space.

For the original population $\mathbf{pop}$, the o-th subspace of the original space is denoted as: $\mathbf{x}^o = (x_1, x_2, ..., x_d)$, where $\mathbf{x}$ is the original variable with D dimensions, d is a random number representing the number of variables in the subspace. The matrix including upper and lower bounds for each original decision variables can be denoted as: $\mathbf{Lu} = (U_1, U_2, ..., U_j, ..., U_D; L_1, L_2, ..., L_j, ..., L_D)$, where U_j is the upper bound corresponding to x_j, L_j is the lower bound corresponding to x_j.

The m-th subspace of the mapping space is denoted as: $\mathbf{y}^m = (y_1, y_2, ..., y_d)$, where $\mathbf{y}$ represents the variable with D dimensions after the PCA mapping. N individuals like $\mathbf{x}^o$ or $\mathbf{y}^m$ make up the population $\mathbf{pop}_x{}^o$ or $\mathbf{pop}_y{}^m$ of a subspace. The coresponding optimization problems of subspace are as shown in Equations (12a) and (12b).

$$\min \hat{f}(\mathbf{x}^o), \tag{12a}$$

$$\min \hat{h}(\mathbf{y}^m), \tag{12b}$$

where $\hat{f}(\mathbf{x}^o)$ and $\hat{h}(\mathbf{y}^m)$ are the fitness values of $\mathbf{x}^o$, $\mathbf{y}^m$ approximated by the corresponding surrogate models.

In the process of updating the global optimal solution once, we only choose either the mapping space or the original space, and K subspaces will be constructed in this space for search. Next, the methods of constructing subspaces will be introduced respectively according to the types of spaces.

For the mapping space, SADE-AMSS constructs a mapping matrix based on the current population distribution. Then, the algorithm will map the current population, the training set for the surrogate model $\hat{h}(\mathbf{y}^m)$, and the boundary matrix into the mapping space. After mapping, the variables corresponding to large eigenvalues and some randomly selected variables together form a subspace. In a few cases, the subspace consists only of the variables corresponding to large eigenvalues. The details of subspace construction based on PCA in the mapping space can be found in Algorithm 1.

Fristly, SADE-AMSS calculates the covariance matrix of the population after removing the mean value in the original space, then calculates the eigenvalues and eigenvectors of the covariance matrix. The eigenvectors are sorted in the descending order of corresponding eigenvalues as the eigenvector matrix $\mathbf{vec}$. Secondly, SADE-AMSS maps the population, the training set, the boundary matrix to the new space. Next, the algorithm checks the size relationship between the upper and lower bounds in the matrix $\mathbf{Lu}_y$ and makes corrections.

Algorithm 1 Mapping subspace construction based on PCA

Input: The current population $\mathbf{pop}$ in the original space; The number of individuals in $\mathbf{pop}$: N; The original training set $\mathbf{X}_{train}$ from the archive Arc; The number of individuals in $\mathbf{X}_{train}$: tsn; The matrix $\mathbf{Lu}$ including upper and lower bounds for each original decision variable; The ordinal number m of the subspace; The number of subspace variables d; α is a positive real number.

Output: The population of the subspace $\mathbf{pop}_y{}^m$; The training set for training the surrogate model $\mathbf{Y}_{train}{}^m$; The matrix $\mathbf{Lu}_y{}^m$ including upper and lower bounds for variables of the subspace; The parameters required for mapping: the average of individual variables in the population $\mathbf{pop}_{mean}$ and eigenvector matrix $\mathbf{vec}$.

if $m == 1$ **then**
  $\mathbf{pop}_{mean} = mean(\mathbf{pop})$.
  $\mathbf{pop} = \mathbf{pop} - \mathbf{pop}_{mean}$. // De-mean.
  $\mathbf{Cov}_{MT} = \mathrm{cov}(\mathbf{pop})$.
  $\mathbf{lambda} = Eigenvalues(\mathbf{Cov}_{MT})$.
  $\mathbf{vec} = Eigenvectors(\mathbf{Cov}_{MT})$.
  $\mathbf{vec} = \mathbf{vec}\{sort(\mathbf{lambda})\}$. // Descending order.
  $d_1 = Count(\mathbf{lambda} > \alpha)$.
  $\mathbf{pop}_y = \mathbf{pop} * \mathbf{vec}$.
  Remove function values of $\mathbf{X}_{train}$ and record.
  $\mathbf{Y}_{train} = \mathbf{X}_{train} - \mathbf{pop}_{mean}, \mathbf{Y}_{train} = \mathbf{Y}_{train} * \mathbf{vec}$.
  $\mathbf{Lu}_y = \mathbf{Lu} - \mathbf{pop}_{mean}, \mathbf{Lu}_y = \mathbf{Lu}_y * \mathbf{vec}$.
  $MakeCorrections(\mathbf{Lu}_y)$.
end if
if $d > d_1$ **then**
  /* Construct the m-th subspace */
  $\mathbf{y}^m = (y_1, y_2, ..., y_{d_1}) \bigcup RandomSelect\{\mathbf{y}_{remain}\}$.
  $\mathbf{y}^m = RandomPermute\{\mathbf{y}^m\}$.
else
  $\mathbf{y}^m = (y_1, y_2, ..., y_d)$.
  $\mathbf{y}^m = RandomPermute\{\mathbf{y}^m\}$.
end if
$\mathbf{pop}_y{}^m = (\mathbf{y}_1{}^m; \mathbf{y}_2{}^m; ...; \mathbf{y}_N{}^m)$.
$\mathbf{Y}_{train}{}^m = (\mathbf{y}_1{}^m; \mathbf{y}_2{}^m; ...; \mathbf{y}_{tsn}{}^m)$.
Add function values recorded by Step 10 to $\mathbf{Y}_{train}{}^m$.
$\mathbf{Lu}_y{}^m = ExtractBounds(\mathbf{y}^m, \mathbf{Lu}_y)$.
return $\mathbf{pop}_y{}^m$; $\mathbf{Y}_{train}{}^m$; $\mathbf{Lu}_y{}^m$; $\mathbf{pop}_{mean}$; $\mathbf{vec}$.

As mentioned above, we pay more attention to the variables mapped by the eigenvectors corresponding to large eigenvalues. These variables should be selected to be optimized, in order to achieve good optimization performance. Taking an individual of the original space after removing the mean value as an example, in a generation, the construction process of two cases based on PCA is shown as Fig. 4, where d is a random number representing the subspace dimension, d_1 is the number of eigenvalues greater than α. The construction of subspaces can be explained in the following two cases respectively.

Case 1: When the required number of subspace variables d is greater than d_1: the first d_1 variables are taken as a part of the subspace; the other $d - d_1$ are randomly selected from the remaining variables; all the selected variables are permuted randomly to form subspace.

Case 2: When the required number of subspace variables d is less than d_1: the first d variables are selected, all the selected variables are permuted randomly to form subspace.

As Step 7) of SADE-AMSS, if the space is the mapping space, before the algorithm goes to the next generation, $\mathbf{pop}_y$ needs to be transformed, and as an initial population $\mathbf{pop}$ of the next generation. The new global optimal solution needs to be transformed to the original space, and evaluated using the exact expensive function. The transformation equations are as follows, take an individual in the mapping space, denote it as $\mathbf{y}$.

$$\mathbf{x} = \mathbf{y} * inv(\mathbf{vec}) + \mathbf{pop}_{mean}, \tag{13}$$

where $inv(\mathbf{vec})$ represents the inverse of the eigenvector matrix $\mathbf{vec}$, $\mathbf{x}$ is the individual corresponding to $\mathbf{y}$ in the original space. It is worth noting that after transforming, we need to check whether the decision variables are out of bounds according to $\mathbf{Lu}$. For the decision variable out of bounds $x_r \in \mathbf{x}$, $1 \le r \le D$, we make the following corrections:

$$\begin{cases} x_r = U_r, & if\ x_r > U_r \\ x_r = L_r, & if\ x_r < L_r \end{cases}, \tag{14}$$

where U_r is the upper bound in $\mathbf{Lu}$ corresponding to x_r, L_r is the lower bound in $\mathbf{Lu}$ corresponding to x_r.

For the original space, SADE-AMSS constructs subspaces as random feature selection in SAEA-RFS. The details of subspace construction based on random decision variable selection in the original space can be found in Algorithm 2.

Algorithm 2 Original subspace construction based on random decision variable selection

Input: The current population $\mathbf{pop}$ in the original space; The number of individuals in $\mathbf{pop}$: N; The original training set $\mathbf{X}_{train}$ from the archive Arc; The number of individuals in $\mathbf{X}_{train}$: tsn; The matrix $\mathbf{Lu}$ including upper and lower bounds for each original decision variable; The ordinal number o of the subspace; The number of subspace variables d.

Output: The population of the subspace $\mathbf{pop}_x{}^o$; The training set for training the surrogate model $\mathbf{X}_{train}{}^o$; The matrix $\mathbf{Lu}_x$ including upper and lower bounds for variables of the subspace.

if $o == 1$ **then**
  $\mathbf{pop}_x = \mathbf{pop}$.
  $\mathbf{Lu}_x = \mathbf{Lu}$.
  Remove function values of $\mathbf{X}_{train}$ and record.
end if
/* Construct the o-th subspace */
$\mathbf{x}^o = RandomSelect\{x_1, x_2, ..., x_D\}$. // D is the original variable dimension.
$\mathbf{x}^o = RandomPermute\{\mathbf{x}^o\}$.
$\mathbf{pop}_x{}^o = (\mathbf{x}_1{}^o; \mathbf{x}_2{}^o; ...; \mathbf{x}_N{}^o)$.
$\mathbf{X}_{train}{}^o = (\mathbf{x}_1{}^o; \mathbf{x}_2{}^o; ...; \mathbf{x}_{tsn}{}^o)$.
Add function values recorded by Step 4 to $\mathbf{X}_{train}{}^o$.
$\mathbf{Lu}_x{}^o = ExtractBounds(\mathbf{x}^o, \mathbf{Lu}_x)$.
return $\mathbf{pop}_x{}^o$; $\mathbf{X}_{train}{}^o$; $\mathbf{Lu}_x{}^o$.

B. Adaptive Search Switching Strategy

SADE-AMSS constructs subspaces in both the original space and the mapping space. Although both subspaces are in low dimensions for modeling the surrogates, they have their own weaknesses. In the original subspaces, each decision variable is selected with the same probability for the optimization process. The population search information is not considered to accelerate the convergence, which can be a waste of expensive computing resources. In addition, since the original subspaces are constructed by random decision variable selection, the probability that all correlated decision variables are grouped together is extremely low. In the mapping subspaces, since the promising directions are extracted based on the past population only, it will lead to the lack of global exploration ability, thus the algorithm is easy to be trapped into local optimum. In order to utilize the advantages of both kinds of subspaces and avoid their disadvantages as much as possible, we need to reasonably assign search resources for them. It is worth noting that as the algorithm proceeds, individuals in the population will gradually concentrate and the population diversity will decrease. Maintaining a good balance between the exploration and the exploitation is very important for both effectiveness and efficiency of the algorithm. To address this challenge, we divide the optimization process into the following three stages.

Stage 1: The global search is carried out in the high-dimensional decision space. In the case of limited computation budget, a local promising area needs to be quickly located to accelerate the convergence process. Therefore, in order to assist the population to shrink rapidly in all directions simultaneously, the surrogate models need to describe the fitness landscape of the whole decision space.

Stage 2: In the found local area, more accurate models need to be used to further exploit. Meanwhile, to increase the diversity, more underexplored areas need to be searched.

Stage 3: The local area has been over exploited. In other words, the surrogate models in the mapping space cannot provide further information, thus the algorithm is easy to trap into local optimum. Therefore, in this stage, the algorithm needs to search unexplored areas.

In order to meet the requirement of both surrogate models and optimization in the different optimization stages, we design a search strategy that quantifies the balance between the exploration and the exploitation for each of these three stages. They are respectively,

1) Str_1: The mapping subspaces are optimized to locate a local promising area to speed up the search, while the original subspaces are used to explore other underexplored areas, we think both kinds of subspaces are of the equal importance. Therefore, the ratio of search resources assigned in the two kinds of subspaces of Str_1 is set to 1:1. That is, in the first generation, K subspaces are constructed sequentially in the original space, in the next generation, K subspaces are constructed sequentially in the mapping space, etc.
2) Str_2: More original subspaces with stronger exploration ability should be optimized, assisted by more accurate models in those. We think the original subspaces is more important than the mapping subspaces, at the same time, we expect to continue to take advantage of the mapping subspaces to speed up the optimization. Therefore, the ratio of search resources assigned in the two kinds of subspaces of Str_2 is set to $Ratio$. That is, in the first $Ratio$ generations, $Ratio * K$ subspaces are constructed sequentially in the original space, in the next generation, K subspaces are constructed sequentially in the mapping space, etc.
3) Str_3: In order to search unexplored areas to jump out of local optimum, we think the original subspaces have a great advantage while the mapping subspaces are not necessarily needed. Hence the ratio of search resources assigned in the two kinds of subspaces of Str_3 is set to 1:0. That is, the subspaces will be only constructed in the original space.

We expect to adopt Str_1 corresponding to Stage 1 in the early stage of the algorithm for locating a local promising area quickly. As the optimization proceeds, if the algorithm traps into a local optimum, it needs to go to the next optimization stage to jump out of local optimum. This means that Str_2 corresponding to Stage 2 or even Str_3 corresponding to Stage 3 will be triggered.

To identify the current convergence state of the algorithm and enable the three search strategies to be switched adaptively, we propose an indicator called DSD to judge the rate of convergence. DSD can be calculated by the following equations:

$$f_t'' = \frac{(\lg f_t - \lg f_{t-1}) - (\lg f_{t-1} - \lg f_{t-2})}{2}, \tag{15}$$

$$E_{mean} = \sum_{i=3}^{t_{es}+3} f_i''/t_{es}, \tag{16}$$

$$\tilde{f}_t = f_t''/E_{mean}, \tag{17}$$

$$DSD(t) = \sum_{j=t-t_r+1}^{t} \tilde{f}_j, \tag{18}$$

where t represents the ordinal number of the current generation, f_t is the best function value up to the t-th generation, t_{es}, t_r represents a pre-set cutoff generation. Equation (16) represents the mean optimization ability of the algorithm in the early stage. It starts with the third generation in order to make sense of f_{t-2} in Equation (15). Equation (17) uses E_{mean} as a scale to calculate the instantaneous optimization ability of the current generation compared with the early stage. Equation (18) translates the instantaneous optimization ability represented by Equation (17) into the optimization ability of the last t_r generations. Besides, in order to make sense of j in Equation (18), the adaptive switching strategy procedures start with generation t_r+2. When Equation (19) is satisfied, i.e., DSD is smaller than a threshold β, that means the convergence rate of the algorithm becomes very slow compared with the early stage. We judge that the convergence curve of the algorithm is close to relatively flat and the algorithm traps into local optimum. To save the function evaluations, we think it is

time to switch the search strategy to enable the optimization to go to the next stage.

$$DSD(t) < \beta. \tag{19}$$

Meanwhile, SADE-AMSS considers that the convergence effect of the algorithm may not be significantly improved within a period of time after switching strategy. Therefore, in order to avoid another switching strategy caused by misjudgment, in $2*t_r+1$ generations after the algorithm switches the strategy, no switch is performed. The above procedures are shown in Algorithm 3.

Algorithm 3 Adaptive search switching strategy

Input: t: the ordinal number of the current generation; E_{mean}: a number calculated by Equation (16); the current strategy flag Str_k; all the contemporary best function values up to the generation t.

Output: The strategy flag Str_k.

1: **if** $t = 1$ **then**
2: $k = 1$, $t_{stop} = 0$. // Set the initial value.
3: **end if**
4: **if** $t \geq t_r+2+t_{stop}$ *and* $k < 3$ **then**
5: Calculate $DSD(t)$ by Equation (18).
6: Check whether Equation (19) is true or false.
7: **if** true **then**
8: $k = k + 1$. // The strategy is set to the next one.
9: $t_{stop} = t + t_r$. // Pause for $2 * t_r + 1$ generations.
10: **end if**
11: **end if**
12: **return** Str_k.

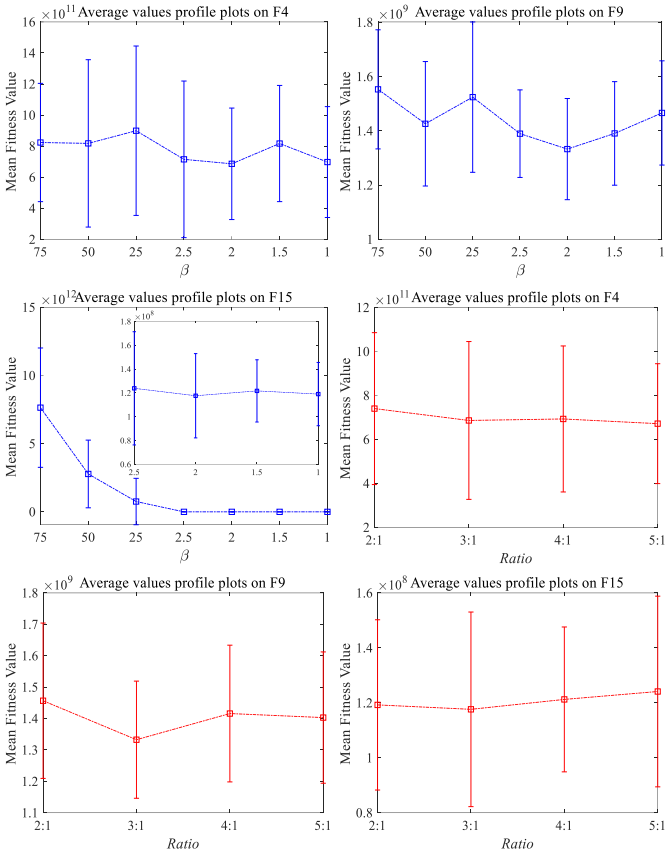


Fig. 5. Average values profile plots over the different thresholds β, the different ratios for Str_2 $Ratio$, in SADE-AMSS on F4, F9, F15 based on 25 independent runs. The blue profile plots corresponds to β, while the red profile plots corresponds to $Ratio$.

IV. Empirical Study

A. Benchmark Functions and Parameter Settings

In order to conduct the experiments, the CEC2010 and CEC2013 benchmark functions [43], [44] are used to test the performance of the compared algorithms. The CEC2010 benchmark functions contain 20 functions and are divided into three categories: fully-separable functions (F1-F3), partially additively separable functions which have different degrees of separability (F4-F18) and non-separable functions (F19, F20). All the functions are 1000-dimensional functions. The CEC2013 benchmark functions are an extension to the existing CEC2010 benchmark functions, which contain 15 functions and are divided into five categories: fully-separable functions (F1-F3), partially additively separable functions with a separable subcomponent (F4-F7), partially additively separable functions with no separable subcomponents (F8-F11), overlapping functions (F12-F14) and non-separable function (F15). F1-F12 and F15 are 1000-dimensional functions, F13 and F14 are 905-dimensional functions.

Some studies have shown that each model has its own advantages and weaknesses for different test problems [45], [46]. [19] mentioned that RBFN appears to have better performance than other surrogate models on the high-dimensional expensive problems. That is why RBFN is adopted for the surrogate model in our algorithm. For the problems of less than 100 dimensions, the accuracy of RBFN is acceptable [19]. Therefore the subspace also contains a maximum of 100 variables, i.e., $1 \leq d \leq 100$. In the initialization, 200 individuals are generated and evaluated using the exact expensive function, then added to the archive Arc, i.e., $N_s = 200$. The number of individuals in the training set tsn is set to $2*d$. The population size N is set to 10, $K = 20$ subspaces will be optimized at each generation. The maximum iterations of subspace optimization is set to 5. DE is used as the optimizer for each subspace, in which $DE/best/1$ and $binomial$ $crossover$ are used as mutation and crossover strategy to generate new individuals. The crossover rate CR is set to 1 and the mutation scaling factor F is set to 0.8. In addition, $t_{es} = 50$, $t_r = 500$, α is set to a positive real number ($\alpha = 1$ in our experiments) such that the first d_1 eigenvalues explain at least η of the total variability (we greedily use $\eta = 99.99\%$ in our experiments) [47]. The threshold β is set to 2. $Ratio$ is set to $3:1$.

Furthermore, three algorithms are compared with our algorithm, which can be summarized as follows.

1) *SAEA-RFS* [19] is a surrogate-assisted optimizer based on the divide-and-conquer strategy. It uses a random feature selection technique to select partial decision variables to form a series of sub-problems. Each sub-problem is optimized assisted by the corresponding surrogate model. SAEA-RFS has good performance in solving large-scale

expensive optimization problems.

2) *FAACSO* [37] uses a fitness approximation strategy to replace most expensive function evaluations. Meanwhile, CSO that solving the large-scale optimization problems efficiently is as the optimizer of the algorithm.
3) *SL-PSO* [20] is a fast optimizer proposed for large-scale optimization.

For a fair comparison, we adopt the recommended settings in the original literature for parameters of the compared algorithms. In all experiments, the stopping criterion of all algorithms is that the maximum number of expensive function evaluations $11*D$ is reached, where D is the search dimension of the problem. Each algorithm will be run independently for 25 times.

B. Sensitivity Analysis

β is a threshold related to the timing of switching strategy in Algorithm 3. $Ratio$ is the ratio of assigning search resources in the two kinds of subspaces for Str_2. Different β and $Ratio$ parameter values may affect the performance of the proposed algorithm. Here, we select three typical functions from the CEC2013 benchmark functions to analyze the sensitivity of β and $Ratio$. They are respectively, a partially additively separable function with a separable subcomponent F4, a partially additively separable function with no separable subcomponents F9, a non-separable function F15.

To evaluate the effects of different values of β, we test a series of values $\beta \in \{75, 50, 25, 2.5, 2, 1.5, 1\}$. The comparison of SADE-AMSS with different β on three functions based on 25 independent runs is presented in Fig. 5. Especially, for F15, the local areas of $\beta \in \{2.5, 2, 1.5, 1\}$ are enlarged for a clear observation. It can be observed that SADE-AMSS with $\beta = 2$ has achieved the best results on all test functions. Moreover, the whole level of $\beta \in \{75, 50, 25\}$ is inferior to $\beta \in \{2.5, 2, 1.5, 1\}$. This might be because when the threshold is large, another search strategy will be performed. However, at the moment, the algorithm does not trap into local optimum, i.e., the advantage of the previous strategy has not been fully exploited. This will lead to a poor performance. Therefore, we keep the threshold as small as possible. In our experiments, β is set to 2 as a general setting for all test instances.

Next, we vary parameter $Ratio$ to examine the effects of different $Ratio$ values. In this set of experiments, $Ratio \in \{2:1, 3:1, 4:1, 5:1\}$ are tested when $\beta = 2$. The experimental results are also shown in Fig. 5. We can see that the results obtained by SADE-AMSS are relatively insensitive to the changes of ratio in strategy Str_2. Overall, the ratio $3:1$ performs best. Hence, we choose $3:1$ as the ratio of assigning search resources in the two kinds of subspaces for Str_2.

C. Performance and Analysis of Different Subspaces

In this section, we compare some variants of SADE-AMSS using different subspaces and SADE-AMSS, to verify the constructional design of multi-subspace in our algorithm. We choose one from each function category of CEC2013 problems as an instance. First, we investigate the effect of mapping subspaces. Here, three variants of SADE-AMSS are adopted for comparison. Concretely, three variants are:

1) SADE-AMSS-Str$_1$ that searches only with strategy Str_1;
2) SADE-AMSS-Str$_2$ that searches only with strategy Str_2;
3) SADE-AMSS-Str$_3$ that searches only with strategy Str_3.

In particular, the first two algorithms search in the mapping subspaces with different proportions, while the last algorithm does not search in the mapping subspaces.

Some average convergence curves of three algorithms based on 25 independent runs on 1000-dimensional F2, F5, F9, F12, F15 are presented in Fig. 6. Note that the local areas of the early stage are enlarged for clear observation. As we expected, for all test functions, it is clear that the mapping subspaces have a huge advantage on convergence in the early stage, which also confirms that the mapping subspaces enable the algorithm to locate a local promising area quicklier than the original subspaces, thus speed up the search. Moreover, for F5 and F9, the convergence curve of SADE-AMSS-Str$_1$ is close to flat gradually in the early stage, i.e., the algorithm traps into local optimum. As metioned in Section III-*B*, the algorithm needs to switch the search strategy adaptively for a greater importance to the original subspaces, so that it can go to the next optimization stage to jump out of local optimum. In addition, for F12, the convergence curve of SADE-AMSS-Str$_1$ is steep during the whole optimization process. Therefore we conclude that the search strategy needs not be switched, which will be tested in the next subsection. For F2 and F15, the convergence curve of SADE-AMSS-Str$_1$ is close to flat in the later stage, in order to achieve better performance, the adaptive switching strategy needs to be performed to jump out of local optimum.

Next, to investigate the effect of original subspaces, the variant that only the mapping space is used to construct the subspaces, denote as SADE-AMSS-Map(pca), is adopted to compare the performance with the proposed algorithm. As shown in Fig. 6, the performance of SADE-AMSS-Map(pca) is worse than the proposed algorithm on all test functions as we expected. In the mapping subspaces constructed by PCA, the promising directions are extracted based on the past population only, it will lead to the lack of global exploration ability, thus the algorithm is easy to be trapped into local optimum. From Fig. 6, it can be seen that SADE-AMSS-Map(pca) achieves the best convergence due to the advantages of mapping subspaces in the early stage. However, the population diversity decreases rapidly, the algorithm traps into local optimum. Therefore, our algorithm considers not only the use of mapping subspaces to accelerate convergence, but also the use of original subspaces to enhance the exploration.

Finally, we will compare a variant with the proposed algorithm. The variant is denoted as SADE-AMSS-Map(rvs), in which only the mapping space is used and random variable selection is used in this space for searching for the optimal solution. As shown in Fig. 6, the performance of SADE-AMSS-Map(rvs) is worse than the proposed algorithm except F12. This variant selects variables randomly only in the mapping space, the advantages of the mapping space for optimization are exploited more in the early stage than the proposed algorithm. However, due to the lack of exploration

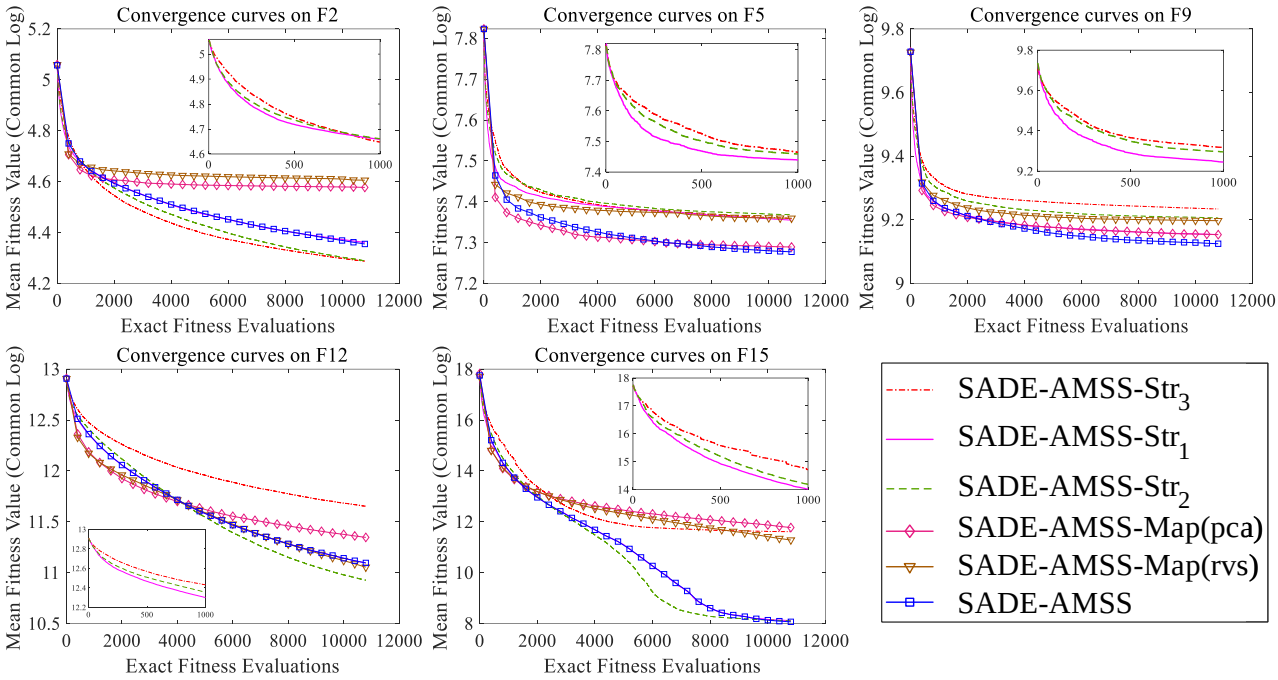


Fig. 6. Convergence curves of SADE-AMSS and five variants of SADE-AMSS on F2, F5, F9, F12, F15.

ability and the small probability of the promising directions being selected, the convergence efficiency of the algorithm gradually decreases, which can be confirmed by Fig. 6. For the overlapping function F12, there is some degree of overlap between the subcomponents, there will be no unique optimal grouping of the decision variables. Therefore, the advantage of random variable selection in the mapping space is signficant, because it means that more groupings will be tried for optimization. The probability of generating groups that contribute to convergence is greatly increased. In addition, in CEC2013, the base function of F12 is not applied with the non-linear transformations, therefore the local promising area is easy to be found, i.e., there are more variables of the mapping space representing the directions that contribute to convergence. Although the algorithm selects variables randomly in the mapping space, the performance may be also good. It can be seen from Fig. 6 that the performance of SADE-AMSS-Map(rvs) is better than the proposed algorithm on F12, which confirms the above conclusions.

D. Effectiveness of Adaptive Switching Strategy

To further discuss the effectiveness of adaptive switching strategy in SADE-AMSS, we adopt two variants of SADE-AMSS to compare with SADE-AMSS. Concretely, two variants are respectively,

1) SADE-AMSS-Str$_1$ that without adaptive switching strategy, 2) SADE-AMSS-Str$_1$-Str$_2$ that only has two search strategies, i.e., Str_1 and Str_2, therefore can only switch the search strategy once adaptively. Meanwhile, in order to better reflect the effectiveness of adaptive switching strategy, the experiment carries out strict control variables. We set up 25 groups of random seeds before the experiment, and each group is used to generate all sequences of pseudo-random numbers in one independent run. We choose one from each function category of CEC2013 problems as an instance except the fully-separable functions, because such functions are relatively simple. Some average convergence curves of three algorithms based on 25 independent runs on 1000-dimensional F5, F9, F12, F15 are presented in Fig. 7. Especially, for F2 and F15, the local areas of the later stage are enlarged for clear observation.

Due to the setting of random seed groups and the same strategy Str_1 of the three algorithms, in the early stage, the convergence curves of the three algorithms coincide on each function, which conforms to the reality. As shown on F5, F9, F15, the algorithm switches the search strategy twice, and it can be seen from the convergence curve that the experimental results are fully in line with expectations. To be specific, the convergence curve of the algorithm that can switch the search strategy twice is the lowest, that is, the convergence effect is the best, followed by the algorithm that can only switch the search strategy once, and finally, the algorithm that without adaptive switching strategy. For F12, the convergence of the algorithm is very good under Str_1, so there is no switching strategy being triggered during the optimization process. Due to the setting of random seed groups and the same strategy Str_1 of three algorithms, the convergence curves of three algorithms coincide, which reflects the adaptive ability of our algorithm. In conclusion, these results verify the conclusion metioned in Section III-*B* that the adaptive switching strategy enables the algorithm to jump out of local optimum and achieve the better convergence effect than before.

Moreover, SADE-AMSS-Str$_2$ that searches only with strategy Str_2 is also adopted to compare with the above three algorithms. As shown in Fig. 7. We can find that the per-

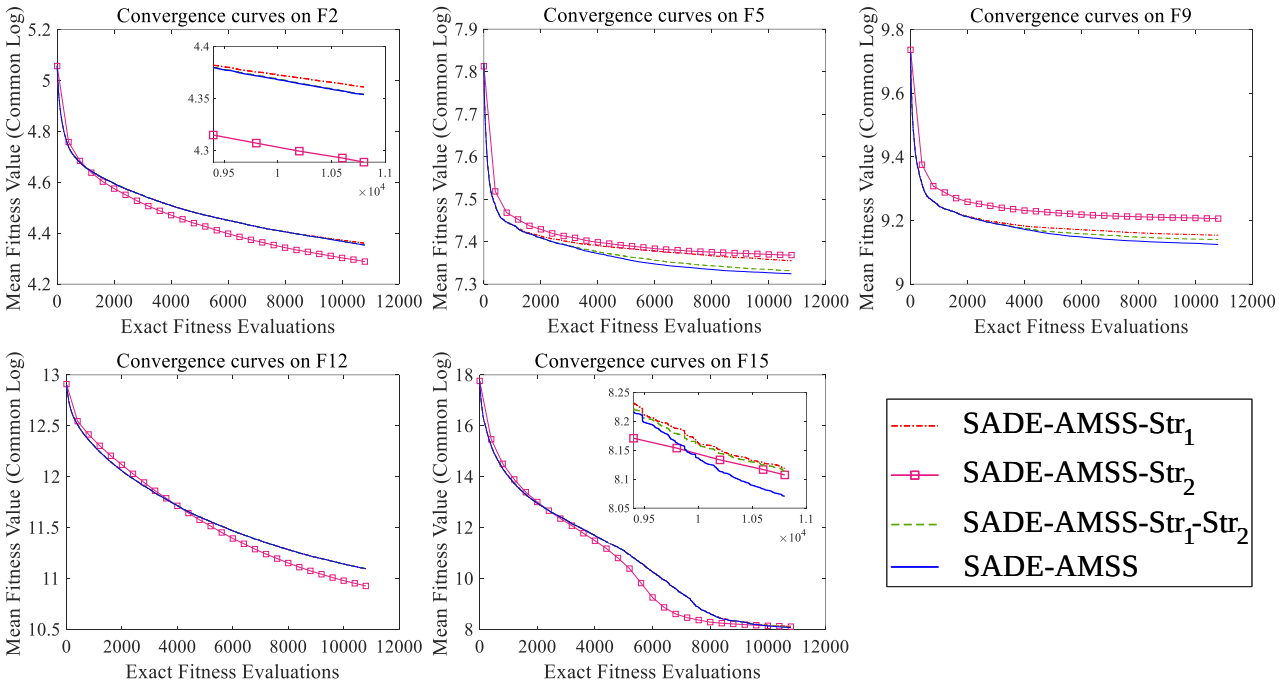


Fig. 7. Convergence curves of SADE-AMSS and three variants of SADE-AMSS on F2, F5, F9, F12, F15.

formance of SADE-AMSS-Str2 is worse than the proposed algorithm except F2 and F12. The search resources assigned in the mapping subspaces of Str_1 are more than those of Str_2. In the early stage, Str_1 will take more advantage of the mapping space to speed up the optimization than Str_2, which can be confirmed from the convergence curves of four algorithms on all test functions in Fig. 7, it is clear that strategy Str_1 has better performance than strategy Str_2 in the early stage. However, for the fully-separable function F2 with a nonlinear transformation, all the decision variables are independent, the advantage of the mapping space for grouping correlated decision variables no longer exists. Furthermore, the destruction of the fitness landscape by nonlinear transformations makes it harder to find a local promising area for the fully-separable problems. We conclude that the advantage of mapping space will diminish quickly. The results in Fig. 7 can confirm the above conclusion. Meanwhile, it can be also seen from Fig. 6, SADE-AMSS-Str3 that have only the original subspaces achieves the best results on F2. Although the advantage of the mapping space decreases rapidly, the convergence curve is still not flat compared to the early stage. That is why our algorithm cannot adaptively switch strategies quickly, which leads to the poor performance of our algorithm on this problem. For F12, the base function is not applied with the non-linear transformations, therefore the local promising area is easy to be found. SADE-AMSS-Str2 with relatively few search resources assigned in the mapping subspaces can also find it in the later stage. With more original subspaces, SADE-AMSS-Str2 explores more underexplored areas, which will result in a satisfactory performance. The above conclusions can be confirmed by the best results obtained by SADE-AMSS-Str2 in Fig. 7.

E. Comparative Experiments

In order to discuss the behavior of the proposed algorithm, we compare SADE-AMSS with SAEA-RFS, FAACSO, SL-PSO on 20 CEC2010 test instances and 15 CEC2013 test instances. The complete results in terms of objective values across 25 independent runs with $11 * D$ function evaluations are tabulated in Tables I and II, where the best results are highlighted. In addition, the Wilcoxon rank-sum test results [48] with the significance level of 0.05 are also presented in the tables. Here, symbol "$\approx$" indicates that there is no statistically significant difference between the results of SADE-AMSS and the compared algorithm, i.e., their performance is almost equivalent. Symbol "$+$" indicates that SADE-AMSS is significantly superior to the compared algorithm, while "$-$" denotes that SADE-AMSS is significantly inferior to the compared algorithm.

From the results on CEC2010 problems, it can be observed that SADE-AMSS obtains the best performance with 13 best results on the 20 test instances, followed by SAEA-RFS with one best result, and FAACSO with six best results. SL-PSO has not achieved the best results among the CEC2010 test problems. To be specific, SADE-AMSS performs better than FAACSO and SL-PSO on 14/20 problems, obtains 17/20 better results than SAEA-RFS.

From the results on CEC2013 problems, it can be observed that SADE-AMSS obtains the best performance with eight best results on the 15 test instances, followed by SAEA-RFS with five best results, and FAACSO with two best results. SL-PSO has not achieved the best results among the CEC2013 test problems. To be specific, SADE-AMSS performs better than FAACSO and SL-PSO on 13/15 problems, obtains 7/15 better results than SAEA-RFS.

TABLE I
STATISTIC RESULTS (MEAN(STANDARD DEVIATION)) OF THE THREE COMPARED ALGORITHMS AND SADE-AMSS ON 20 CEC2010 TEST INSTANCES BY THE WILCOXON RANK-SUM TEST (THE SIGNIFICANCE LEVEL IS 0.05). THE BEST RESULTS FOR EACH INSTANCE ARE HIGHLIGHTED. THE SYMBOLS "+," "−," AND "≈" INDICATE THAT SADE-AMSS IS STATISTICALLY SIGNIFICANTLY SUPERIOR TO, INFERIOR TO, AND ALMOST EQUIVALENT TO THE PARTICULAR COMPARED ALGORITHM, RESPECTIVELY

Problems	SAEA-RFS	FAACSO	SL-PSO	SADE-AMSS
F1	1.4147e+10 (1.7262e+09) +	5.3845e+10 (4.0316e+09) +	6.3451e+10 (3.1672e+09) +	7.8910e+09 (4.8752e+08)
F2	1.3052e+04 (2.2430e+02) +	1.5509e+04 (2.0049e+02) +	1.6041e+04 (1.6643e+02) +	1.1847e+04 (3.3198e+02)
F3	2.0675e+01 (1.0087e-01) +	2.0740e+01 (3.8514e-02) +	2.0912e+01 (3.7994e-02) +	2.0479e+01 (5.7630e-02)
F4	6.5519e+13 (2.4580e+13) ≈	3.4367e+14 (4.5612e+13) +	6.1179e+14 (7.4900e+13) +	5.7033e+13 (1.7562e+13)
F5	5.1539e+08 (5.8482e+07) +	4.0682e+08 (1.9103e+07) −	4.3568e+08 (1.7339e+07) ≈	4.6373e+08 (7.4003e+07)
F6	1.9459e+07 (3.0323e+06) +	9.1499e+06 (6.2019e+05) ≈	1.2263e+07 (4.1232e+05) ≈	1.2815e+07 (6.8962e+06)
F7	2.1510e+10 (4.6145e+09) −	7.2052e+10 (9.1434e+09) +	9.4630e+10 (1.0826e+10) +	2.6538e+10 (8.5160e+09)
F8	7.1284e+11 (2.8051e+12) ≈	2.2815e+14 (7.5799e+13) +	4.0940e+14 (8.6730e+13) +	5.6292e+09 (5.3740e+09)
F9	3.6399e+10 (3.2583e+09) +	5.9342e+10 (3.1810e+09) +	8.6466e+10 (3.3576e+09) +	1.2445e+10 (9.7414e+08)
F10	1.8706e+04 (4.9798e+02) +	1.5543e+04 (2.6203e+02) −	1.6174e+04 (2.0946e+02) −	1.6883e+04 (4.5541e+02)
F11	2.3478e+02 (4.2699e-01) +	2.2166e+02 (1.1656e+00) −	2.2459e+02 (8.7361e-01) −	2.3381e+02 (4.5086e-01)
F12	5.7541e+06 (4.7323e+05) +	7.3797e+06 (4.1952e+05) +	1.0027e+07 (5.5894e+05) +	3.0228e+06 (1.8094e+05)
F13	6.4548e+10 (1.7243e+10) +	3.3396e+11 (2.8216e+10) +	4.1338e+11 (1.8593e+10) +	6.5613e+09 (2.5716e+09)
F14	5.4721e+10 (4.6143e+09) +	7.3661e+10 (3.2340e+09) +	1.0160e+11 (6.2165e+09) +	1.5758e+10 (1.4755e+09)
F15	2.0385e+04 (6.0692e+02) +	1.5558e+04 (1.9360e+02) −	1.6231e+04 (2.1100e+02) −	1.9241e+04 (5.0926e+02)
F16	4.2796e+02 (4.6116e-01) +	4.1228e+02 (1.3181e+00) −	4.1601e+02 (9.2030e-01) −	4.2714e+02 (4.6663e-01)
F17	1.0283e+07 (8.0803e+05) +	1.4890e+07 (5.8379e+05) +	2.0972e+07 (1.1854e+06) +	4.2930e+06 (4.4938e+05)
F18	6.9660e+11 (7.0184e+10) +	1.2008e+12 (6.4914e+10) +	1.4431e+12 (6.5439e+10) +	9.7775e+10 (2.7452e+10)
F19	2.5767e+07 (3.1396e+06) +	3.7931e+07 (3.2425e+06) +	5.2331e+07 (6.5239e+06) +	1.7889e+07 (2.5443e+06)
F20	7.6774e+11 (7.5912e+10) +	1.4778e+12 (7.6247e+10) +	1.6724e+12 (8.0513e+10) +	1.0064e+11 (1.2048e+10)
$+/\approx/-$	17/2/1	14/1/5	14/2/4	

From the experimental results on 35 test instances, it can be observed that none of the algorithms is able to successfully handle all types of problems. Specifically, the surrogate-assisted algorithms (SAEA-RFS and SADE-AMSS) beat the non-surrogate-assisted algorithms (FAACSO and SL-PSO) by a strong advantage except F5, F6, F10, F11, F15, F16 in CEC2010 functions, and F5, F9 in CEC2013 functions. This may be because the prediction accuracy of RBFN for these functions is too low to take advantage of the unique advantages of the surrogate mechanism. In terms of overall performance, we can conclude that the surrogate mechanism has better performance on large-scale expensive optimization than other methods.

For the two surrogate-assisted algorithms, SADE-AMSS is significantly superior or almost equivalent to SAEA-RFS except F7 in CEC2010 functions and F1, F2, F3, F6 in CEC2013 functions. For the fully-separable functions F1, F2, and F3 in CEC2013 functions, except for applying several nonlinear transformations to the base functions, the design does not differ from that of CEC2010 functions. We can conclude that our algorithm is a little weak on the fully-separable functions with some nonlinear transformations. However, as mentioned above, the fully-separable functions such as F1, F2, and F3 in CEC2013 functions, are very simple. Therefore, this result is acceptable, our algorithm has more advantages to be applied to solve the complex real-world industrial engineering optimization problems than SAEA-RFS. Moreover, we can find that the performance of our proposed algorithm becomes better when the problem is less separable. This might be because when the problem is less separable, the mapping space in SADE-AMSS can more reflect the advantages of grouping as mentioned at the beginning of Section III, which makes optimization more effective.

For the two non-surrogate-assisted algorithms, the performance of FAACSO is greater than SL-PSO. That is reasonable, because in SL-PSO, the expensive nature of function evaluations are not considered. Meanwhile FAACSO uses a fitness approximation strategy as a consideration for the expensive nature of function evaluations.

In conclusion, SADE-AMSS optimizes the mapping subspaces to speed up the optimization process. Meanwhile, three adaptive search strategies are designed for jumping out of local optimum. The experimental results indicate that our algorithm is more competitive than existing methods for large-scale expensive optimization.

V. CONCLUSION

In this work, we propose an SADE-AMSS algorithm for large-scale expensive optimization. The algorithm constructs subspaces in the mapping space based on the information of past population search. With the surrogate models that contain the information of the whole decision space, the algorithm quickly locates a local promising area, thus the convergence speed is increased greatly. According to the optimization stage, an adaptive search switching strategy is proposed to keep a good balance between the exploration and the exploitation.

The experimental results on the CEC2010 problems and CEC2013 problems show that SADE-AMSS achieves much better performance than three compared algorithms given the same number of expensive function evaluations, especially on the less separable problems. This work proves the effectiveness of PCA mapping subspaces. For large-scale expensive

TABLE II
STATISTIC RESULTS (MEAN(STANDARD DEVIATION)) OF THE THREE COMPARED ALGORITHMS AND SADE-AMSS ON 15 CEC2013 TEST INSTANCES BY THE WILCOXON RANK-SUM TEST (THE SIGNIFICANCE LEVEL IS 0.05). THE BEST RESULTS FOR EACH INSTANCE ARE HIGHLIGHTED. THE SYMBOLS "+," "−," AND "≈" INDICATE THAT SADE-AMSS IS STATISTICALLY SIGNIFICANTLY SUPERIOR TO, INFERIOR TO, AND ALMOST EQUIVALENT TO THE PARTICULAR COMPARED ALGORITHM, RESPECTIVELY

Problems	SAEA-RFS	FAACSO	SL-PSO	SADE-AMSS
F1	7.4686e+09 (4.1358e+09) −	5.2764e+10 (3.3591e+09) +	6.3427e+10 (3.0550e+09) +	8.1492e+09 (6.3163e+08)
F2	1.9389e+04 (1.1013e+03) −	3.4100e+04 (9.4639e+02) +	3.8381e+04 (9.4048e+02) +	2.2646e+04 (6.7223e+02)
F3	2.0906e+01 (1.3665e-02) −	2.1626e+01 (8.1815e-03) +	2.1632e+01 (5.0375e-03) +	2.1002e+01 (1.9944e-02)
F4	1.1712e+12 (4.5710e+11) +	2.3853e+12 (3.0104e+11) +	3.8138e+12 (5.2914e+11) +	6.8637e+11 (3.5896e+11)
F5	2.2864e+07 (4.8939e+06) +	1.3226e+07 (1.1327e+06) −	1.4998e+07 (6.7263e+05) −	1.8937e+07 (2.8375e+06)
F6	1.0604e+06 (3.3986e+03) −	1.0663e+06 (1.5424e+03) +	1.0668e+06 (1.1238e+03) +	1.0627e+06 (2.0553e+03)
F7	7.6305e+09 (2.8999e+09) ≈	1.5904e+12 (4.4785e+11) +	9.7734e+11 (3.5383e+11) +	6.1484e+09 (2.4149e+09)
F8	2.5103e+16 (1.8518e+16) ≈	5.6640e+16 (9.0522e+15) +	1.2789e+17 (3.1336e+16) +	2.2094e+16 (1.6178e+16)
F9	1.7143e+09 (3.8587e+08) +	1.0172e+09 (4.8351e+07) −	1.1698e+09 (3.2753e+07) −	1.3327e+09 (1.8650e+08)
F10	9.4397e+07 (6.0918e+05) ≈	9.5047e+07 (4.9514e+05) +	9.5383e+07 (2.5153e+05) +	9.4590e+07 (5.1358e+05)
F11	1.0231e+12 (4.3721e+11) +	2.3109e+14 (7.8307e+13) +	2.3183e+14 (7.0345e+13) +	6.4072e+11 (2.8979e+11)
F12	4.4963e+11 (3.9628e+10) +	1.5068e+12 (6.9886e+10) +	1.6945e+12 (8.8791e+10) +	1.2471e+11 (2.8281e+10)
F13	9.5676e+10 (3.1865e+10) ≈	1.0671e+12 (2.9935e+11) +	2.3410e+14 (7.0023e+13) +	9.5521e+10 (2.0397e+10)
F14	1.4187e+12 (4.1797e+11) +	8.9677e+12 (2.1479e+12) +	2.9933e+14 (8.0397e+13) +	1.1717e+12 (4.1995e+11)
F15	4.1055e+11 (1.8770e+12) +	4.4998e+14 (8.8472e+13) +	3.9926e+14 (5.0104e+13) +	1.1765e+08 (3.5403e+07)
+/ ≈ /−	7/4/4	13/0/2	13/0/2	

optimization, it is promising that modeling surrogates and searching in the PCA mapping subspaces can accelerate the convergence.

In our work, we use linear combinations of decision variables as the promising directions by PCA. However, the nonlinear combination of decision variables can also express the search directions. How nonlinear combinations can be used to express those promising directions will be our future work. In addition, we will investigate the adaptive adjustment strategy for the parameter β rather than using one value during the algorithm process. Furthermore, we will try to apply the method of multi-subspace search to large-scale expensive multi/many-objective optimization.

REFERENCES

[1] Komarudin and A. Chandra, "Optimization of very large scale capacitated vehicle routing problems," in *Proceedings of the 2019 5th International Conference on Industrial and Business Engineering*, 2019, pp. 18–22.

[2] M. Erdem and S. Bulkan, "A two-stage solution approach for the large-scale home healthcare routeing and scheduling problem," *South African Journal of Industrial Engineering*, vol. 28, no. 4, pp. 133–149, 2017.

[3] N. R. Sabar, A. Turky, A. Song, and A. Sattar, "An evolutionary hyper-heuristic to optimise deep belief networks for image reconstruction," *Applied Soft Computing*, vol. 97, p. 105510, 2020.

[4] T. Jansson, L. Nilsson, and M. Redhe, "Using surrogate models and response surfaces in structural optimization with application to crashworthiness design and sheet metal forming," *Structural & Multidisciplinary Optimization*, vol. 25, no. 2, pp. 129–140, 2003.

[5] D. Douguet, "e-lea3d: a computational-aided drug design web server," *Nucleic acids research*, vol. 38, no. suppl_2, pp. W615–W621, 2010.

[6] Y. Jin and B. Sendhoff, "A systems approach to evolutionary multiobjective structural optimization and beyond," *IEEE Computational Intelligence Magazine*, vol. 4, no. 3, pp. 62–76, 2009.

[7] Y. Jin, H. Wang, and C. Sun, *Data-Driven Evolutionary Optimization*. Springer Nature Switzerland AG, 2021.

[8] T. Back, *Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms*. Oxford university press, 1996.

[9] R. Cheng and Y. Jin, "A competitive swarm optimizer for large scale optimization," *IEEE Transactions on Cybernetics*, vol. 45, no. 2, pp. 191–204, 2015.

[10] Y. Sun, M. Kirley, and S. K. Halgamuge, "A recursive decomposition method for large scale continuous optimization," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 647–661, 2017.

[11] Z. Yang, T. Ke, and Y. Xin, "Multilevel cooperative coevolution for large scale optimization," in *Proceedings of the IEEE Congress on Evolutionary Computation, CEC 2008, June 1-6, 2008, Hong Kong, China*, 2008.

[12] Y. Qiang, W. N. Chen, J. D. Deng, L. Yun, and J. Zhang, "A level-based learning swarm optimizer for large-scale optimization," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 99, pp. 578–594, 2018.

[13] Y. F. Ge, W. J. Yu, Y. Lin, Y. J. Gong, and J. Zhang, "Distributed differential evolution based on adaptive mergence and split for large-scale optimization," *IEEE Transactions on Cybernetics*, vol. 48, no. 99, pp. 2166–2180, 2018.

[14] M. A. Potter and K. A. D. Jong, "A cooperative coevolutionary approach to function optimization," in *International Conference on Parallel Problem Solving from Nature*. Springer, 1994, pp. 249–257.

[15] R. Salomon, "Reevaluating genetic algorithm performance under coordinate rotation of benchmark functions," *Biosystems*, vol. 39, no. 3, pp. 263–278, 1960.

[16] M. N. Omidvar, X. Li, Y. Mei, and X. Yao, "Cooperative co-evolution with differential grouping for large scale optimization," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 3, pp. 378–393, 2014.

[17] Y. Davidor, "Epistasis variance: Suitability of a representation to genetic algorithms," *Complex Systems*, vol. 4, no. 4, pp. 369–383, 1990.

[18] I. D. Falco, A. D. Cioppa, and G. A. Trunfio, "Investigating surrogate-assisted cooperative coevolution for large-scale global optimization," *Information Sciences*, vol. 482, 2019.

[19] G. Fu, C. Sun, Y. Tan, G. Zhang, and Y. Jin, "A surrogate-assisted evolutionary algorithm with random feature selection for large-scale expensive problems," in *International Conference on Parallel Problem Solving from Nature*. Springer, 2020, pp. 125–139.

[20] R. Cheng and Y. Jin, "A social learning particle swarm optimization algorithm for scalable optimization," *Information Sciences*, vol. 291, pp. 43–60, 2015.

[21] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. IEEE, 1995, pp. 1942–1948.

[22] Y. Tang, J. Chen, and J. Wei, "A surrogate-based particle swarm optimization algorithm for solving optimization problems with expensive black box functions," *Engineering Optimization*, vol. 45, no. 5, pp. 557–576, 2013.

[23] D. Buche, N. Schraudolph, and P. Koumoutsakos, "Accelerating evolutionary algorithms with gaussian process fitness function models," *IEEE Transactions on Systems Man & Cybernetics Part C*, vol. 35, no. 2, pp. 183–194, 2005.

[24] R. G. Regis, "Evolutionary programming for high-dimensional constrained expensive black-box optimization using radial basis functions," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 3, pp. 326–347, 2014.

[25] Y. Jin, M. Olhofer, and B. Sendhoff, "A framework for evolutionary optimization with approximate fitness functions," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 5, pp. 481–494, 2002.

[26] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[27] Y. Jin, "A comprehensive survey of fitness approximation in evolutionary computation," *Soft computing*, vol. 9, no. 1, pp. 3–12, 2005.

[28] S. Liu, H. Wang, W. Peng, and W. Yao, "A surrogate-assisted evolutionary feature selection algorithm with parallel random grouping for high-dimensional classification," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2022.

[29] Z. Song, H. Wang, C. He, and Y. Jin, "A kriging-assisted two-archive evolutionary algorithm for expensive many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 6, pp. 1013–1027, 2021.

[30] T. Zhang, H. Wang, B. Yuan, Y. Jin, and X. Yao, "Surrogate-assisted evolutionary q-learning for black-box dynamic time-linkage optimization problems," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2022.

[31] N. Zheng, H. Wang, and B. Yuan, "An adaptive model switch-based surrogate-assisted evolutionary algorithm for noisy expensive multi-objective optimization," *Complex & Intelligent Systems*, pp. 1–18, 2022.

[32] Y. Yuan and W. Banzhaf, "Expensive multiobjective evolutionary optimization assisted by dominance prediction," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 1, pp. 159–173, 2022.

[33] W. Wang, H.-L. Liu, and K. C. Tan, "A surrogate-assisted differential evolution algorithm for high-dimensional expensive optimization problems," *IEEE Transactions on Cybernetics*, pp. 1–13, 2022.

[34] F. Li, X. Cai, L. Gao, and W. Shen, "A surrogate-assisted multiswarm optimization algorithm for high-dimensional computationally expensive problems," *IEEE Transactions on Cybernetics*, vol. 51, no. 3, pp. 1390–1402, 2021.

[35] X. Wang, G. G. Wang, B. Song, P. Wang, and Y. Wang, "A novel evolutionary sampling assisted optimization method for high-dimensional expensive problems," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 815–827, 2019.

[36] S. Rahnamayan and S. J. Mousavirad, "Towards solving large-scale expensive optimization problems efficiently using coordinate descent algorithm," in *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2020, pp. 2506–2513.

[37] C. Sun, J. Ding, J. Zeng, and Y. Jin, "A fitness approximation assisted competitive swarm optimizer for large scale expensive optimization problems," *Memetic Computing*, vol. 10, no. 2, pp. 123–134, 2018.

[38] J. Park and I. Sandberg, "Universal approximation using radial-basis-function networks," *Neural Computation*, vol. 3, no. 2, pp. 246–257, 1991.

[39] H. Yu, Y. Tan, J. Zeng, C. Sun, and Y. Jin, "Surrogate-assisted hierarchical particle swarm optimization," *Information Sciences*, vol. 454-455, pp. 59–72, 2018.

[40] H. M. Gutmann, "A radial basis function method for global optimization," *Journal of Global Optimization*, vol. 19, no. 3, pp. 201–227, 2001.

[41] J. Shlens, "A tutorial on principal component analysis," *International Journal of Remote Sensing*, vol. 51, no. 2, 2014.

[42] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011.

[43] T. Ke, X. Li, P. N. Suganthan, Z. Yang, and T. Weise, "Benchmark functions for the cec 2010 special session and competition on large-scale global optimization," *nature inspired computation & applications laboratory*, 2010.

[44] X. Li, K. Tang, M. N. Omidvar, Z. Yang, and K. Qin, "Benchmark functions for the cec 2013 special session and competition on large-scale global optimization," *Gene*, vol. 7, no. 33, p. 8, 2013.

[45] D. Zhao and D. Xue, "A comparative study of metamodeling methods considering sample quality merits," *Structural and Multidisciplinary Optimization*, vol. 42, no. 6, pp. 923–938, 2010.

[46] R. Jin, W. Chen, and T. W. Simpson, "Comparative studies of metamodelling techniques under multiple modelling criteria," *Structural and Multidisciplinary Optimization*, vol. 23, no. 1, pp. 1–13, 2001.

[47] K. Antonov, E. Raponi, H. Wang, and C. Doerr, "High dimensional bayesian optimization with kernel principal component analysis," *arXiv preprint arXiv:2204.13753*, 2022.

[48] J. D. a, S. G. b, D. M. C, and F. H. A, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm & Evolutionary Computation*, vol. 1, no. 1, pp. 3–18, 2011.